COP 3502C Programming Assignment # 5

Binary Search Trees

Please read all the pages before starting to write your code

Please include the following commented lines at the beginning of your code to declare your authorship of the code:

/* COP 3502C PA5

This program is written by: Your Full Name */

Compliance with Rules: *The UCF Golden Rules apply to this assignment and its submission. In addition, all assignment policies outlined in the syllabus also apply here. The instructor or TA may ask any student to explain any part of their code to verify authorship and ensure clarity. Sharing this assignment description or your code, in whole or in part, with anyone or anywhere is a violation of course policy. Likewise, using code obtained from any outside source, including AI tools, will be considered cheating. Your primary resources should be class notes, lectures, labs, and TAs. These rules are in place to ensure fairness, maintain academic integrity, and give every student the opportunity to develop their own problem-solving and coding skills.*

Deadline:

Please check Webcourses for the official assignment deadline. Assignments submitted by email will not be graded, and such emails will not receive a response in accordance with course policy.

What to do if you need clarification on the problem?

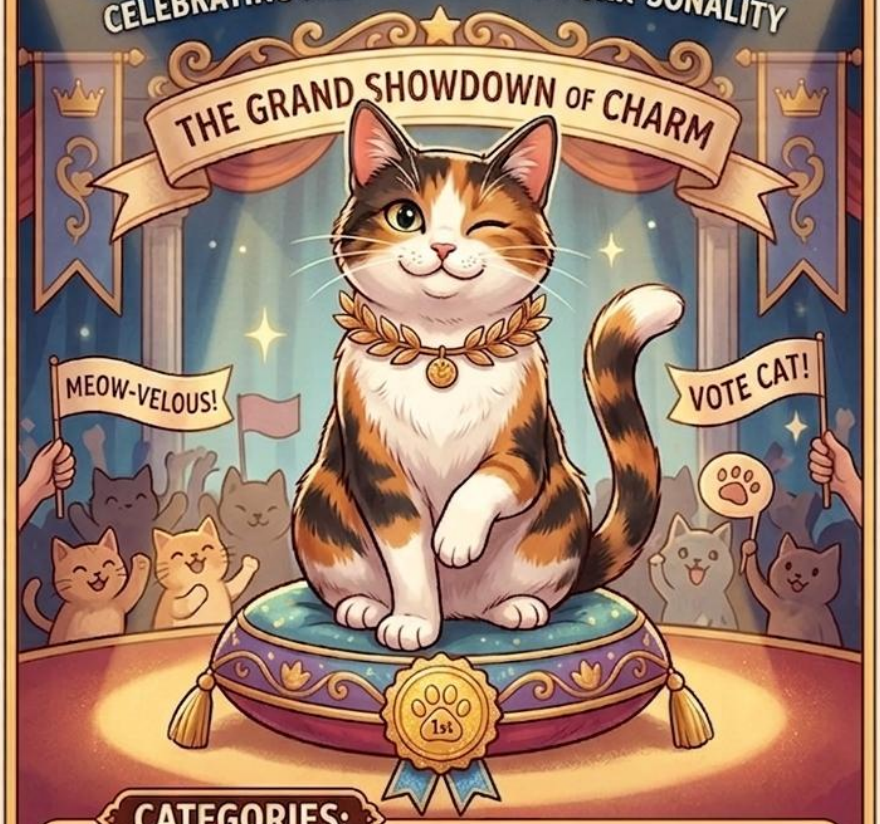
If you need clarification on the problem, a discussion thread will be created on Webcourses, and you are strongly encouraged to post your questions there. You may also ask questions on Discord; if TAs are available, they can respond, and other students might also contribute, helping you get answers more quickly. For further clarification on the requirements, you are welcome to visit the TAs or the instructor during office hours.

How to get help if you're stuck?

As stated in the course policy, office hours are the primary place to seek help. While we may occasionally respond to emails, debugging requests are best addressed during office hours rather than through email.

CAT CHARM TOURNAMENT

CELEBRATING FELINE FINESSE & PURR-SONALITY



CATEGORIES:

FRIENDLY | GRUMPY | PLAYFUL | LAZY | CURIOUS

WHEN & WHERE:

CUTEST MEOW | PURR-SONALITY PLUS | DATE: TBA | VENUE: CAT HALL OF FAME

REGISTER YOUR CHARMER TODAY!

WWW.CATCMERYOURCHARMER.COM

Objective:

- Give practice with implementing standard binary search tree functions.
- Give practice maintaining subtree data within insertion and deletion operations.
- Writing recursive functions as counters and checkers
- Apply insertion, deletion and search with given information.
- Practice designing queries for BST structures.

Background story:

The Annual Cat Charm Tournament is hosting their competition. Each cat competes and receives a unique charm score from judges. The organizing committee wants to maintain a BST as the live leaderboards. Cats register, get disqualified, and the committee runs different tests to see which are valid candidates within the competition!

You've designed this program to do the following:

- Efficiently insert and remove cats from the leaderboard
- Maintain subtree metadata
- Look up any cat's standing or find a current rank
- Analyze breed diversity across score brackets
- Mass disqualifies cats based on trait conditions

Develop a program that manages the competition leaderboard using a binary search tree, supporting real-time updates and complex queries.

Problem Description:

You will take in a sequence of operations representing the events during competition. Cats are registered and inserted into a binary search tree using their **name as the key**. Cats may be disqualified and removed at any point through the competition. During this process, the organizing committee will issue queries against the leaderboard to determine rankings and filter your data.

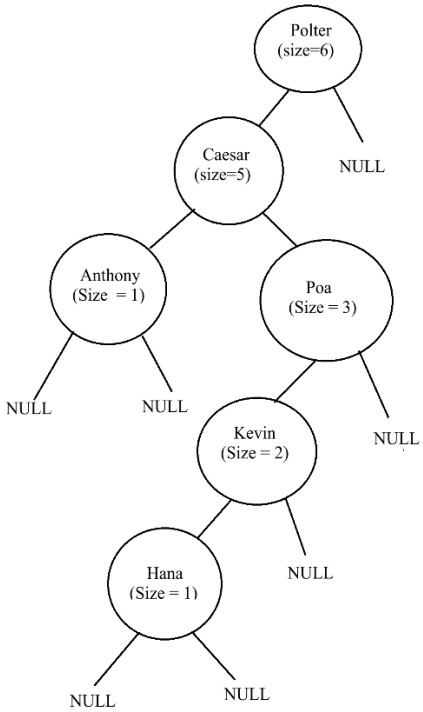
Variables and Context:

The program takes input representing a sequence of operations for the competition leaderboard. Each operation either modifies the tree or queries it for information.

Each cat's information is stored as `Cat` struct that holds a dynamically allocated name and breed, an integer charm score, and a statically allocated array of 5 trait flags. The name serves as the tree search key. Each insertion / search should use strcmp() to navigate the tree. The charm score is data within the node but does **not** determine the position.

Each trait flag corresponds to an index in the `TRAIT_NAMES` constant array: `friendly [0], grumpy [1], playful [2], lazy [3], curious [4].` A value of 1 means the cat possesses that trait; 0 means it does not.

Each node in the tree is stored as a `BST_Node` structure that holds the data `Cat`, left and right child pointers. The `subtree_size` represents the total number of nodes in that node's subtree (not the height). For the example below, a tree can look like this (size represents the `subtree_size`):



`Polter` has `subtree_size = 6` because there are 6 nodes in its subtree (including itself). `Caesar` has `subtree_size = 5`, `Anthony` has `subtree_size = 1`, and so on. If a cat is deleted, every ancestor's subtree_size along the path must be decremented accordingly. `subtree_size` can be computed using the sum of the `left & right subtree_size + 1` as each `subtree_size` is inclusive of its own node

*Note: The tree will **not** be balanced. Its shape depends entirely on the insertion order.*

Required Data Structures:

```
#define MAX_NAME 25 // max name of a cat
#define NUM_TRAITS 5 // number of total traits a cat has

typedef struct Cat {
    char *name; //dynamically allocate space for a cat's name
    char *breed; //dynamically allocate space for a cat's breed
    int charm; //unique charm score for each cat
    int traits[NUM_TRAITS]; //scores corresponding to each index in TRAIT_NAMES
} Cat;
```

```
typedef struct BST_Node {
    Cat *cat;
    struct BST_Node *left;
    struct BST_Node *right;
    int subtree_size; // stores the current size of this subtree's root (not height)
} BST_Node;

const char *TRAIT_NAMES[NUM_TRAITS] =  {"friendly", "grumpy", "playful", "lazy", "curious"};
```

Queries Explained:

- ***Query One:*** This query follows the insertion for the given tree. During this process, you should use the name of the cat as a means for organizing your tree. During insertion, if a duplicate name is found, the existing node's data (breed, charm, and traits) is replaced only if the new cat has strictly more traits set to 1. Otherwise, the insertion is ignored.

- ***Query Two:*** This follows deletion with the three-case approach taught during lecture. For deleting it will just use a cat's name and try to search for the necessary node. It is not guaranteed that during this process a node will be found.

Please note (IMPORTANT): to keep your output consistent with ours, when deleting the node who's a parent of two children, please utilize selecting the successor (minimum of the right) node as necessary.

- ***Query Three:*** This query finds the k-th smallest element in the tree. You will be given a value k, and your task is to locate the k-th smallest node based on the tree's in-order ordering. For example, if k is equal to 1, you would return the smallest element in the tree. If k is equal to 3, you would return the third smallest element. This operation should make use of the subtree sizes stored in each node to efficiently locate the target element.

- ***Query Four:*** This query requires implementing a function that filters cats based on a specific trait. The function will receive a trait index and a trait value as input. Using this information, you must create an array of strings containing the names of cats that match the given criteria, sorted in alphabetical order.

  To do this, iterate through the cats in alphabetical order. For each cat, access its trait array and check whether the value at the specified index matches the given trait value. If it does, add the cat's name to the result array.

  Refer to the implementation requirements for details about the function parameters.

- ***Query Five:*** Given a trait index and a trait value, remove any node that has these constraints. If the values were 0 1, respectively, delete all cats who have a value of 1 at index 0 for their given traits array. This query will then output the number of nodes removed.
- ***Query Six:*** Goes through all elements in the tree by printing via in-order traversal showing subroot sizes for each node in the tree.

Inputs & Outputs Explained:

The first line of input, a single positive integer, $\boldsymbol{n}$ $(1 \leq n \leq 10^6)$, represents the number of operations for the input case. Each of the next n lines will be a single positive integer, q ($1 \leq \boldsymbol{q} \leq 6$).

Due to how the auto-grader is set up, please ensure to leave no trailing spaces in any of the outputs and end with a '\n' character.

When q = 1

Input:

```
1 <name> <breed> <charm> <t0> <t1> <t2> <t3> <t4>
```

The remaining tokens on the line are used to create a node and insert it into the binary search tree keyed on the cat's name in alphabetical order. The name and breed are strings of at most 25 characters. The charm value is an integer. Each node t0, t1, …, t4 correspond to the respective index in `TRAIT_NAMES`. All values in the node array is either set to 0 or 1.

If a cat with the same name already exists in the tree, the existing node's data (breed, charm, and traits) is replaced only if the new cat has strictly more traits set to 1. Otherwise, the insertion is ignored.

Output:

After successfully entering a new node, "`Insert: <depth>`" where depth resembles here the node sits height wise from the root of the tree. For instance, the root has a depth of zero. If a node was replaced, then "`Replaced`" will be the output.

When q = 2

Input:

```
2 <name>
```

The token after q will be a string representing the name of the cat to be deleted. It is **NOT** guaranteed that this name exists in the tree at the time of this operation and should be terminated early when possible.

Output:

Regardless of the outcome for this query please print the following statement:

```
Deletion Complete
```

***Please note*: to keep your output consistent with ours, when deleting the node who's a parent of two children, please utilize selecting the successor (minimum of the right) node as necessary.**

When q = 3

Input:

```
3 <value>
```

The token following the query number is a positive integer K.

Given this integer, find the K-th smallest element in the tree based on alphabetical ordering. If K = 1, return the cat whose name comes first alphabetically. If K = 3, return the cat ranked 3rd alphabetically. If K is larger than the number of cats in current tree, then print **`"NO SMALLEST ELEMENT FOUND"`** Otherwise, print the cat's name, breed, and charm score.

Output:

Input: `3 1 1` Output: `Anthony Tabby 91`

When q = 4

Input:

```
4 <traitIndex> <traitValue>
```

The next two tokens will represent a `trait index` $(0 \leq traitIndex \leq 4)$ and a `trait value` either 0 or 1. This uses DMA (malloc char** to tree size); deep copy names print all cats with the trait listed. **Please refer to the implementation requirements for this query.**

Output:

If no cats match the given trait condition, print "NONE FOUND" on a line by itself. Otherwise, print the trait name followed by a colon, then all matching cat names separated by spaces on a single line. For example, if cats Hana, Coco, and Kevin are friendly, the output would be:

```
friendly: Coco Hana Kevin
```

When q = *5*

Input:

```
5 <traitIndex> <traitValue>
```

The next two tokens will be integers representing a `trait index` (0 ≤ traitIndex ≤ 4) and a `trait value` (0 or 1).

Output:

If no cats match the given trait condition, print "`NONE REMOVED`" on a line by itself. Otherwise, print the count of removed cats as a single integer on a line by itself.

When q = *6*

Input:

```
6
```

There are no additional tokens on the line.

Output:

For each query of type 6, if the tree is empty, print "`EMPTY`" on a line by itself. Otherwise, print each node's name, charm score, and subtree_size separated by spaces, one node per line, in alphabetical order (ascending)

Sample Program Run:

Example Input (standard input)	Example Output (standard output)
20 1 Polter Ragdoll 85 1 0 1 0 1 1 Caesar Siamese 47 0 1 0 1 0 1 Poa Persian 72 1 1 0 0 1 1 Kevin Maine 33 0 0 1 1 0 1 Anthony Tabby 91 1 0 0 1 1 1 Hana DSH 68 0 0 1 0 0 6 3 1 3 4 4 0 1 4 3 1 2 Polter 6 5 0 1 6 2 Caesar 2 Hana 2 Kevin 6 3 1	Insert: 0 Insert: 1 Insert: 2 Insert: 3 Insert: 2 Insert: 4 Anthony 91 1 Caesar 47 5 Hana 68 1 Kevin 33 2 Poa 72 3 Polter 85 6 Anthony Tabby 91 Kevin Maine 33 friendly: Anthony Poa Polter lazy: Anthony Caesar Kevin Deletion Complete Anthony 91 1 Caesar 47 5 Hana 68 1 Kevin 33 2 Poa 72 3 2 Caesar 47 3 Hana 68 1 Kevin 33 2 Deletion Complete Deletion Complete Deletion Complete EMPTY NO SMALLEST ELEMENT FOUND

Implementation Requirements/Run Time Requirements *[Please read it carefully and make sure to fulfill these requirements]*

You will be required to make a binary search tree where each node is of type BST_Node. You must only add a new node when the name does not already exist in the tree. If the name already exists, replacement logic applies (see query1). You must only structurally delete a node when query 2 or query 5 is called.

Note: We will check this by looking at your code not its execution. It's absolutely possible to get the correct output without following our implementation specifications, but a decent portion of the grade will come from following the implementation specifications.
Your deletion code must match the 3 cases approach including case 2.1 and case 2.2 as explained in the class and must match the code shown in the class/recording. Implementing any other ways will result in -100 in your PA5.
If your program times out, this is most likely due to improper traversal of the tree.

Query 1, 2, 3 **O(h)**:

Your code must follow O(h) time for inserting and deleting as needed to maintain the purpose of this structure. Ideally, you should start from the root and get to the correct node and insert or delete properly. along with this be sure to keep track of subtree_size. No full tree traversal is permitted.

While linear solutions do exist, it's important to utilize the logarithmic properties of a tree since this is what sets this data structure unique to others, so please ensure to follow these runtimes.

Query 4 **O(n)**:

This query required traversing the entire tree. Below you must implement the following function prototype:

```
char** filterByTrait(BST_Node *root, int traitIndex, int traitValue, int *resultSize)
```

Initially, you should malloc a char** of size equal to the total number of nodes in the tree. Perform a deep copy for each node that satisfies this query and malloc for the exact string length with no wasted space. Before returning this char**, you must not have any wasted space for this and should realloc to the exact size.

Query 5 **O(n * h)**:

This query involves two distinct phases. The first phase requires traversing the tree to identify which nodes satisfy the condition whilst the second is identifying which node satisfies the condition. Ideally, these two phases are kept separate.

Query 6 **O(n)**:

This query requires a full in-order traversal of the tree. During this traversal, each node's name, charm score, and subtree_size should be printed on its own line. If the tree is empty. Then print "EMPTY"

Other standard requirements:

- Do not use global variables at all. You may have globally declared constants.
- Your code must compile and execute on Codegrade to receive any credit.
- Your code should not have any memory leaks to receive full credit.
- Make sure to use good variable names, put comments on important blocks of code, and indent your code properly.
- We will test your code with large test cases, and your code must run within the time limit set on Codegrade. Common reasons to time out include unnecessary loops, less optimized code, unnecessary traversal, or unnecessary sorting.
- Use DMA for storing strings in the structs, following the standard approach from class and allocating exact memory needed for the string size.
- Note that this code does not require any sorting algorithm

Hints and Recommended Approach

- Start by implementing insert and search using strcmp for comparisons.
- Verify your tree is correctly ordered alphabetically before moving on to delete and query logic
- Test deletion thoroughly with all three cases
- Use query 5 to your advantage to ensure values are being updated properly as some of the other queries are reliant on this information

Deliverables:

- Please submit source file, `main.c`, via Webcourses->Codegrade.